Functions in C

1. Introduction

In programming, a **function** is a block of code that performs a **specific task**.

Functions help to **divide a large program into smaller, manageable parts**.

Instead of writing the same code many times, we can place the code inside a **function** and call it whenever needed.

This approach is called **modular programming**.

Example Without Functions

```
#include <stdio.h>

int main()
{
   int a = 5, b = 3;

   printf("Sum = %d\n", a + b);

   int c = 10, d = 6;

   printf("Sum = %d\n", c + d);

   return 0;
}
```

The addition logic is repeated.

Example Using a Function

```
#include <stdio.h>

int add(int x, int y)
{
   return x + y;
}

int main()
{
   printf("Sum = %d\n", add(5,3));
   printf("Sum = %d\n", add(10,6));

   return 0;
}
```

Now the **addition logic is written only once**.

Advantages of Functions

Functions provide several benefits:

1. **Code reusability** – functions can be used many times.
2. **Modularity** – programs become easier to organize.
3. **Better readability** – programs are easier to understand.
4. **Easier debugging** – errors can be located quickly.
5. **Reduces repetition** – avoids writing the same code multiple times.

Structure of a Function

A function has three main parts:

1. **Function declaration (prototype)**
2. **Function definition**
3. **Function call**

1. Function Declaration (Prototype)

A **function declaration** tells the compiler about the function before it is used.

Syntax

return_type function_name(parameters);

Example

int add(int a, int b);

This tells the compiler:

- Function name: **add**
- Returns: **integer**
- Takes **two integers as parameters**

2. Function Definition

This is the **actual implementation of the function**.

Syntax

```
return_type function_name(parameters)
{
   // statements
}
```

Example

```
int add(int a, int b)
{
   int sum = a + b;
   return sum;
}
```

3. Function Call

A function is executed when it is **called from another function**, usually main().

Example

```
result = add(5,3);
```

Complete Example

```
#include <stdio.h>

int add(int a, int b);

int main()
{
   int result;

   result = add(4,6);

   printf("Sum = %d", result);

   return 0;
}

int add(int a, int b)
{
   return a + b;
}
```

Types of Functions in C

Functions can be classified into two main types:

1. **Library Functions**
2. **User-defined Functions**

1. Library Functions

These are **built-in functions provided by C libraries**.

Examples include:

Function	Purpose
printf()	Output
scanf()	Input
sqrt()	Square root
strlen()	String length
pow()	Power

Example

```
printf("Hello");
```

2. User-Defined Functions

These are functions **created by the programmer**.

Example

```
int multiply(int a, int b)
{
   return a * b;
}
```

Categories of User-Defined Functions

Functions can be classified based on **parameters and return values**.

Type	Description
1	No parameters, no return value
2	Parameters, no return value
3	No parameters, return value
4	Parameters and return value

1. Function with No Parameters and No Return Value

```
#include <stdio.h>

void greet()
{
   printf("Welcome to C Programming\n");
}

int main()
{
   greet();
   return 0;
}
```

2. Function with Parameters but No Return Value

```
#include <stdio.h>

void displaySum(int a, int b)
{
   printf("Sum = %d", a + b);
}

int main()
{
   displaySum(5,3);
   return 0;
}
```

3. Function with Return Value but No Parameters

```
#include <stdio.h>

int getNumber()
{
   int num;
   printf("Enter number: ");
   scanf("%d", &num);
   return num;
}

int main()
{
   int number = getNumber();
   printf("You entered %d", number);

   return 0;
}
```

4. Function with Parameters and Return Value

This is the **most commonly used type**.

```
#include <stdio.h>

int square(int n)
{
   return n * n;
}

int main()
{
   int result = square(5);

   printf("Square = %d", result);

   return 0;
}
```

Function Parameters

Parameters are **variables used to receive values** passed to a function.

Example

```
int add(int a, int b)
```

Here:

- a and b are parameters.

Return Statement

The return statement:

- Sends a value back to the calling function.
- Ends the function execution.

Example

```
return result;
```

Scope of Variables

Variables declared inside a function are called **local variables**.

Example

```
void test()
{
   int x = 10;
}
```

x cannot be used outside the function.